

```

# -----
# Nash Bargaining
# -----

import os
import sys

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, NonlinearConstraint, Bounds

# Printing the outputs as a text file if ==1
output_file = 0

# -----
# Problem Set-up
# -----
def fun(x):
    return -x[0] * x[1] # negative

def constr(x):
    return x[0]**2 + x[1]**2

# nonlinear constraint:  $x_1^2 + x_2^2 \leq 1 \Leftrightarrow -\infty \leq \text{constraint\_fun}(x) \leq 1$ 
nonlcon = NonlinearConstraint(constr, -np.inf, 1.0)

bounds = Bounds([0.0, 0.0], [np.inf, np.inf]) # lb = [0; 0], ub = []
x0 = np.array([0.1, 0.1])

# -----
# minimize (trust-constr)
# -----
result = minimize(fun, x0, method='trust-constr',
                  constraints=[nonlcon], bounds=bounds)

x = result.x
fval = result.fun
lam_nonlcon = result.v[0] # multiplier for  $x_1^2 + x_2^2 \leq 1$ 
lam_bounds = result.v[1] # multipliers for  $[x_1 \geq 0, x_2 \geq 0]$ 

if output_file == 1:
    dfile = "Nash_bargaining_python.txt"
    if os.path.exists(dfile):
        os.remove(dfile)
    sys.stdout = open(dfile, "w")

# -----
# Output
# -----
print(f"Nash Bargaining Solution: {x[0]:.4f},{x[1]:.4f}")
print(f"Multiplier (non-linear const): {lam_nonlcon[0]:.5f}")
print(f"Multiplier (non-negativity const): {lam_bounds[0]:.5f}, {lam_bounds[1]:.5f}")

if output_file == 1:
    sys.stdout.close()
    sys.stdout = sys.__stdout__

```

```

# -----
# Plot
# -----
slope_const = -x[0] / x[1] # slope of the constraint tangent at the solution
x1_lin = np.linspace(0, 1, 100)
x2_lin = np.linspace(0, 1, 100)
x1_mesh, x2_mesh = np.meshgrid(x1_lin, x2_lin)

fig, ax = plt.subplots()
ax.grid(True)

# constraint boundary  $x_1^2 + x_2^2 = 1$ 
ax.contour(x1_mesh, x2_mesh, x1_mesh**2 + x2_mesh**2, levels=[1],
           colors='k', linewidths=1)
ax.contour(x1_mesh, x2_mesh, x1_mesh * x2_mesh, levels=[-fval],
           colors='b', linewidths=1) # indifference curve
ax.contour(x1_mesh, x2_mesh, x1_mesh * x2_mesh,
           levels=[0.75 * (-fval), 1.25 * (-fval)],
           colors='k', linewidths=1, linestyle='--') # neighboring indifference curves
# tangent line to the constraint at the optimum
ax.plot(x1_lin, slope_const * (x1_lin - x[0]) + x[1], 'k', linewidth=1)
ax.plot(x[0], x[1], 'ro', linewidth=1) # optimal point
ax.set_xlabel(r'$x_{1}$', fontsize=12)
ax.set_ylabel(r'$x_{2}$', fontsize=12)
ax.set_title('Nash Bargaining Solution', fontsize=14)
ax.set_xlim([0, 1])
ax.set_ylim([0, 1])
plt.show()

```